



VIT[®]
B H O P A L
www.vitbhopal.ac.in

STUDENT MANAGEMENT & ACADEMIC RECORD SYSTEM

Project Report: SMARS

Submitted By: Kanak Gupta

Programming Language: Java

Development Environment: Visual Studio Code

Java Version: JDK 26

Academic Year: 2026-2027

TABLE OF CONTENTS

1. Introduction
2. Problem Statement
3. Objectives
4. Scope of the Project
5. Functional Requirements
6. Non-Functional Requirements
7. System Architecture
8. System Workflow
9. UML Diagrams
10. Modules Description
11. Technologies Used
12. Implementation
13. Testing and Results
14. Screenshots
15. Challenges Faced
16. Learnings
17. Future Enhancements
18. Conclusion
19. References

INTRODUCTION

Student Management & Academic Record System (SMARS) is a Java-based console application developed to manage student academic information in an organized and efficient manner.

The system allows users to manage student details, course information, marks, grades, attendance and academic reports.

The project demonstrates the practical implementation of Core Java and Object-Oriented Programming concepts such as classes and objects, encapsulation, ArrayList, methods, conditional statements, loops, input validation, exception handling and file handling.

The application provides a simple menu-driven interface through which users can perform different student and academic record management operations.

PROBLEM STATEMENT

Managing student academic records manually can be time-consuming and may lead to errors in data entry, calculation and record management.

Maintaining student details, course information, marks and attendance separately can also make searching and updating records difficult.

Therefore, there is a need for a simple computer-based system that can organize student academic information and perform common academic calculations automatically.

The Student Management & Academic Record System (SMARS) provides a structured solution for managing student records, courses, marks, grades, attendance and academic reports.

OBJECTIVES

- To manage student personal information.
- To add, view, search, update and delete student records.
- To manage course information.
- To record marks for different subjects.
- To automatically calculate total marks and percentage.
- To automatically calculate student grades.
- To determine pass/fail status.
- To record and calculate attendance percentage.
- To generate complete academic reports.
- To validate user input.
- To save important records using text files.
- To demonstrate practical use of Java and OOP concepts.

SCOPE

The scope of SMARS includes the management of student personal information, courses, marks, grades, attendance and academic reports.

The system provides CRUD operations for students and courses. It also performs automatic academic calculations and provides input validation.

The current version is a console-based Java application using text files for data storage.

Target Users

- Students
- Teachers
- College/Department Staff
- Academic Administrators

FUNCTIONAL REQUIREMENTS

MODULE	FUNCTION
Student Management	Add, View, Search, Update, Delete
Course Management	Add, View, Search, Update, Delete
Marks Management	Add marks, Calculate total, Calculate percentage
Grade Management	Calculate grade, Pass/Fail
Attendance Management	Record attendance, Calculate percentage, Eligibility
Report Management	Generate and save academic report
Validation	Validate user inputs
File Management	Save student, course and report data

NON-FUNCTIONAL REQUIREMENTS

Performance

The system should provide quick responses for normal student and course operations.

Usability

The system provides a simple menu-driven console interface that is easy to understand.

Reliability

Input validation and exception handling are used to reduce common application errors.

Maintainability

The project is divided into multiple classes and modules, making the system easier to understand and maintain.

Error Handling

Invalid inputs such as incorrect marks, invalid phone numbers and invalid IDs are handled using validation methods.

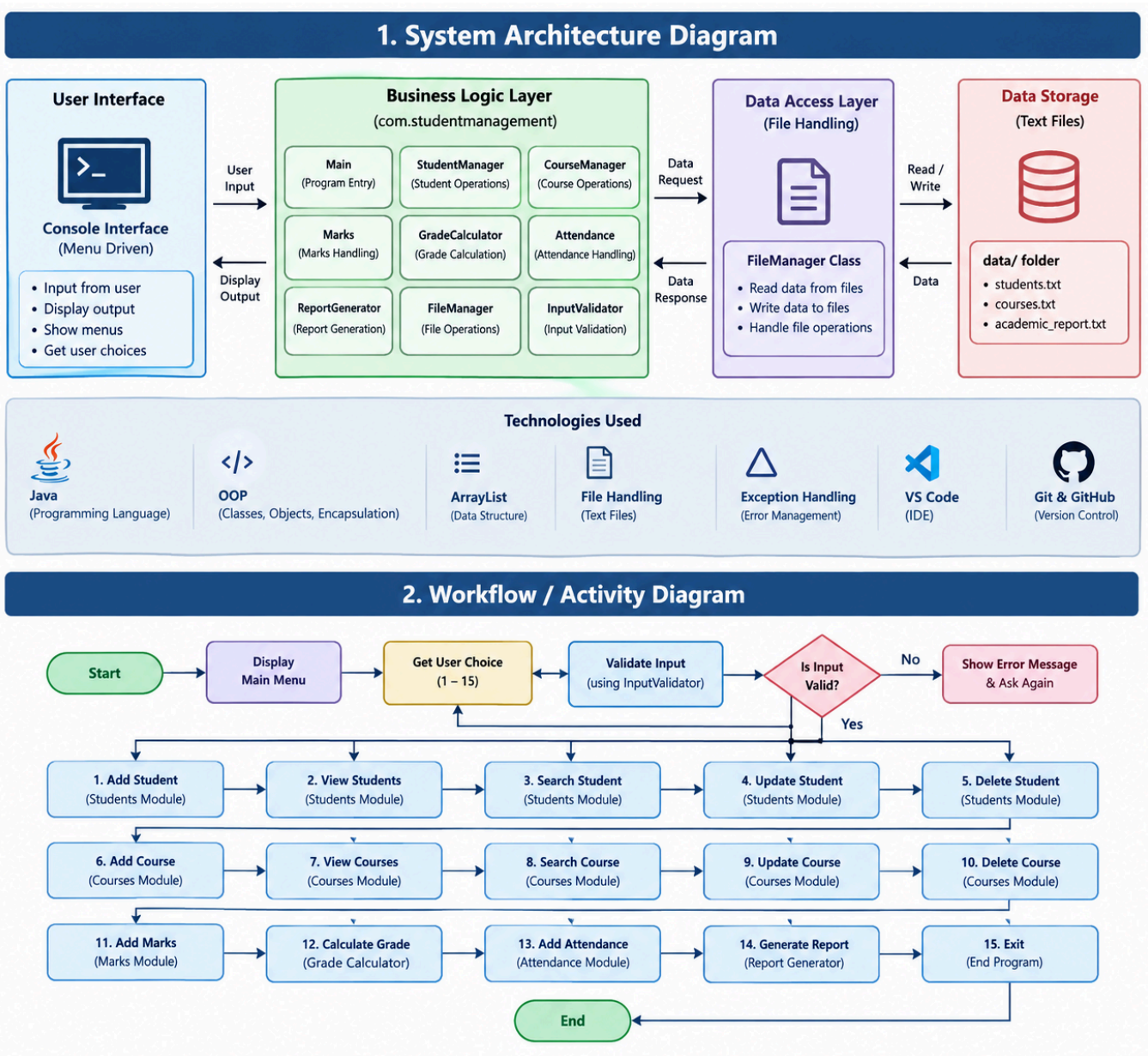
Resource Efficiency

The application uses lightweight Java collections and text files for data storage.

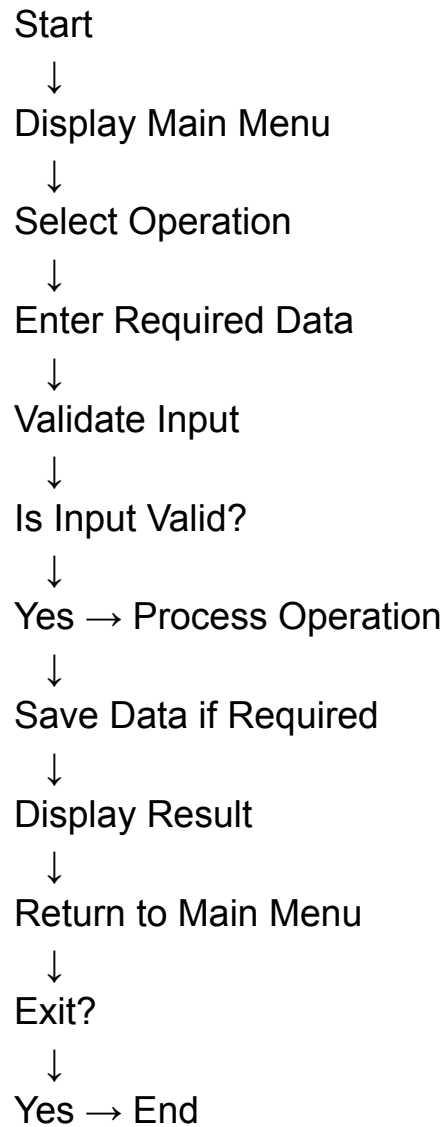
SYSTEM ARCHITECTURE

The system follows a modular architecture where different classes perform specific responsibilities. Main.java controls the overall application flow, manager classes handle operations, model classes store data, utility classes provide validation and file handling, and ReportGenerator generates academic reports.

Architecture + Workflow Diagram



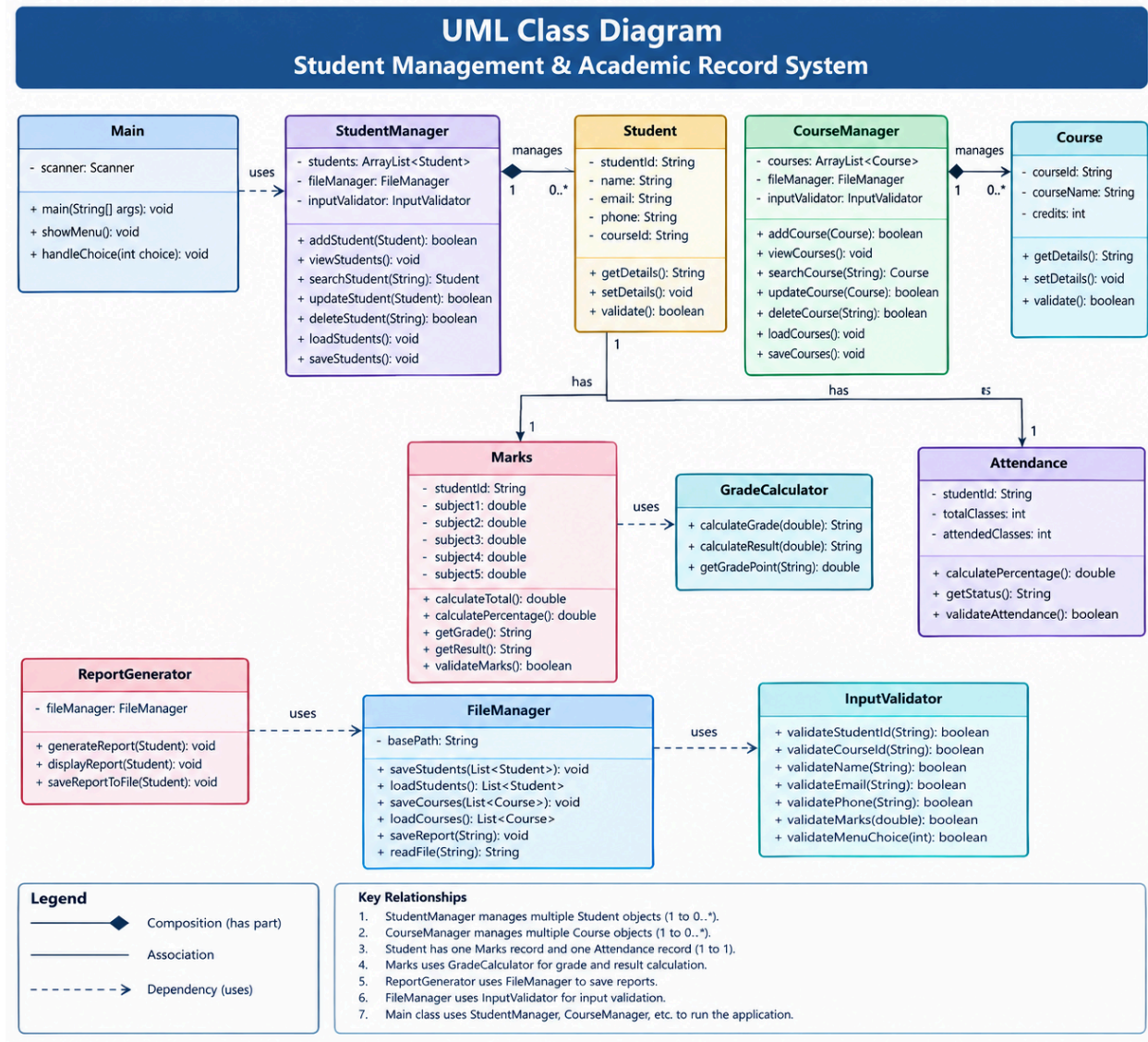
WORKFLOW



The application starts by displaying the main menu. The user selects an operation and enters the required information. The system validates the input before processing it. If the input is valid, the selected operation is performed and the result is displayed. Data can also be saved into text files. The system continues until the user selects the Exit option.

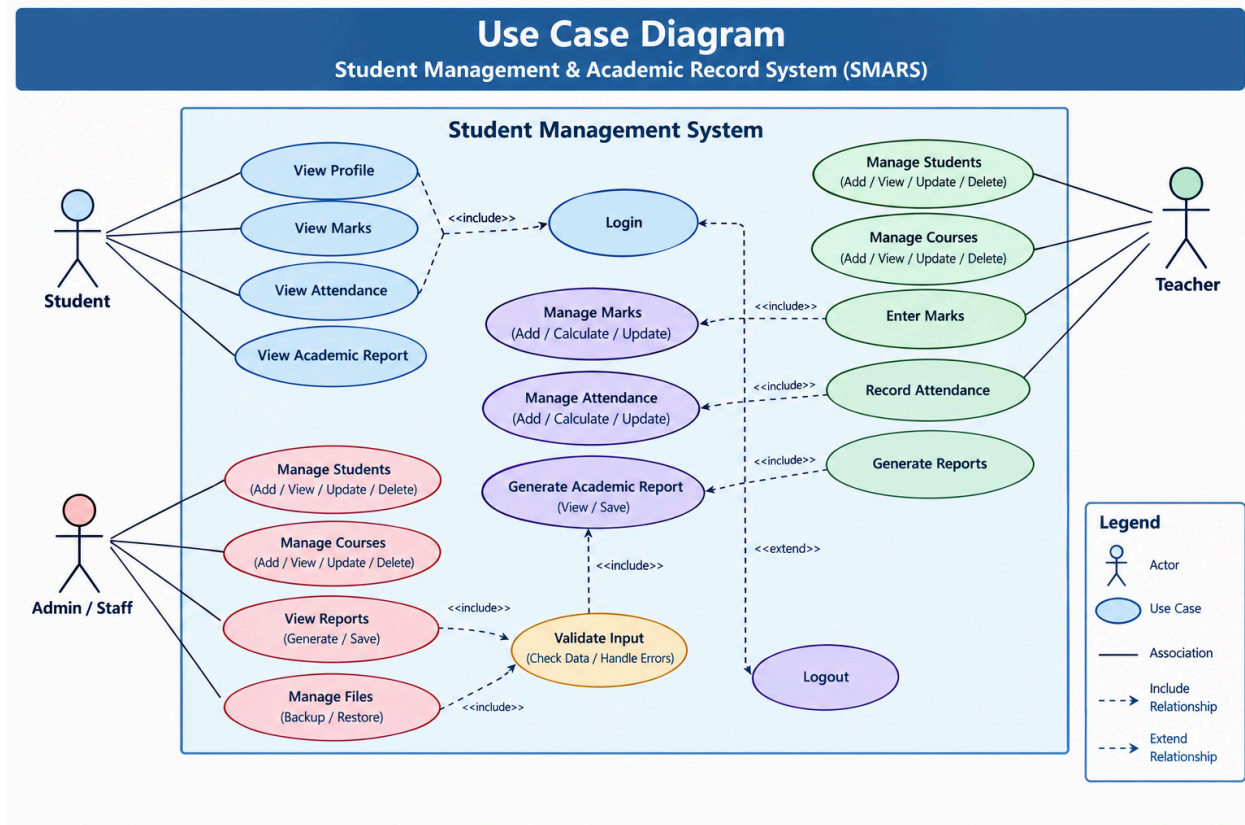
UML DIAGRAMS

Class Diagram: The Class Diagram represents the major classes used in the Student Management & Academic Record System and shows their responsibilities and relationships.



Use Case Diagram: The Use Case Diagram represents the interaction

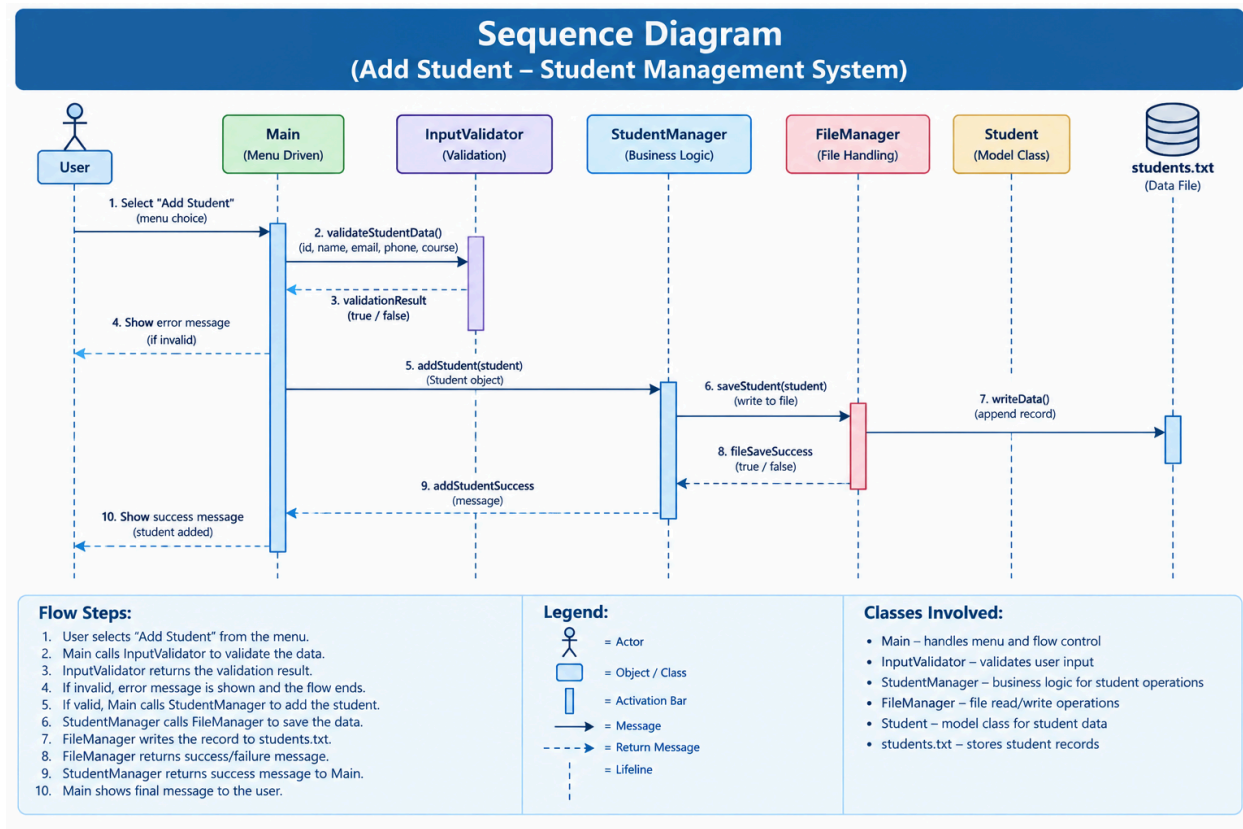
between users and the major functions of the Student Management System.



Sequence Diagram: The Sequence Diagram represents the sequence of interactions involved in adding a student to the system.

The user selects the Add Student option. Main receives the input and InputValidator validates the data. After successful validation, StudentManager adds the student and FileManager saves the student

record into the data file.



MODULES

1. Student Management Module

Handles adding, viewing, searching, updating and deleting student records.

2. Course Management Module

Handles course information including course ID, course name, instructor and credits.

3. Marks Management Module

Stores marks of five subjects and calculates total marks and percentage.

4. Grade Management Module

Calculates the grade and pass/fail result based on percentage.

5. Attendance Management Module

Records total and attended classes and calculates attendance percentage and eligibility.

6. Report Management Module

Generates a complete academic report containing student details, marks, grade and attendance.

7. File Management Module

Saves student, course and academic report data into text files.

8. Input Validation Module

Checks whether the data entered by the user is valid.

TECHNOLOGIES + IMPLEMENTATION

Technologies used:

Technology	Purpose
Java	Main programming language
JDK 26	Java Development Kit
Visual Studio Code	Development environment
ArrayList	Store objects in memory
File Handling	Store data in text file
Git/GitHub	Version control and project submission

Implementation:

- `Main.java` — Controls menu and application flow.
- `Student.java` — Stores student information.
- `StudentManager.java` — Performs student operations.
- `Course.java` — Stores course information.
- `CourseManager.java` — Performs course operations.
- `Marks.java` — Stores marks and calculates percentage.
- `GradeCalculator.java` — Calculates grade and result.
- `Attendance.java` — Calculates attendance.
- `ReportGenerator.java` — Generates academic report.
- `FileManager.java` — Handles file storage.
- `InputValidator.java` — Validates user input.

TESTING

Test Case	Expected Result	Status
Add Student	Student added successfully	PASS
View Student	Student details displayed	PASS
Search Student	Correct student found	PASS
Update Student	Student details updated	PASS
Add Course	Course added successfully	PASS
View/Search Course	Course details displayed	PASS
Update Course	Course details updated	PASS
Add Marks	Marks and total calculated	PASS

Calculate Grade	Grade and result displayed	PASS
Add Attendance	Attendance calculated	PASS
Generate Report	Complete report generated	PASS
Exit	Application terminates	PASS

SCREENSHOTS

Main Menu:

```
=====
      STUDENT MANAGEMENT SYSTEM
=====
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student

6. Add Course
7. View Courses
8. Search Course
9. Update Course
10. Delete Course

11. Add Marks
12. Calculate Grade
13. Add Attendance
14. Generate Student Report

15. Exit
=====
```

Student Management:

```
Enter your choice: 1
Enter Student ID: 5604
Enter Student Name: Kanak Gupta
Enter Email: kanakgupta@gmail.com
Enter Phone: 1234567891
Enter Course: btech CSE
Student added successfully!
Student data saved successfully!
```

Marks:

```
Enter your choice: 11
Enter Student ID: 5604
Enter Subject 1 Marks: 85
Enter Subject 2 Marks: 90
Enter Subject 3 Marks: 78
Enter Subject 4 Marks: 88
Enter Subject 5 Marks: 92

Marks added successfully!
Total      : 433.0
Percentage : 86.6%
```

Grade:

```
Enter your choice: 12

===== GRADE DETAILS =====
Percentage : 86.6%
Grade      : A
Result     : PASS
```

Attendance:

```
Enter your choice: 13
Enter Student ID: 5604
Enter Total Classes: 40
Enter Attended Classes: 34

Attendance added successfully!
Attendance : 85.00%
Status     : ELIGIBLE
```

Academic Report:

```
=====
                        STUDENT ACADEMIC REPORT
=====

----- STUDENT DETAILS -----
Student ID : 5604
Name       : Kanak Gupta
Email      : kanakgupta@gmail.com
Phone      : 2345678912
Course     : btech CSE

----- MARKS DETAILS -----
Subject 1  : 85.0
Subject 2  : 90.0
Subject 3  : 78.0
Subject 4  : 88.0
Subject 5  : 92.0
Total      : 433.0
Percentage : 86.6%
Grade      : A
Result     : PASS

----- ATTENDANCE DETAILS -----
Total Classes : 40
Attended Classes : 34
Attendance     : 85.00%
Status        : ELIGIBLE

=====
                        END OF REPORT
=====

Academic report saved successfully!

=====
```

CHALLENGES + LEARNINGS

Challenges Faced

- Managing multiple Java classes and their relationships.
- Connecting different modules with the main menu.
- Implementing input validation.

- Handling incorrect user inputs.
- Implementing file handling.
- Generating a complete academic report.

Learnings

- Learned practical implementation of Object-Oriented Programming.
- Learned how to use classes and objects.
- Learned to use ArrayList for storing objects.
- Learned input validation techniques.
- Learned Java file handling.
- Learned how to divide a project into multiple modules.
- Learned testing and debugging of a Java application.

FUTURE ENHANCEMENTS + CONCLUSION

Future Enhancements

- MySQL database integration.
- Login and authentication system.
- Graphical User Interface.
- Subject-wise attendance.
- PDF report generation.
- Automatic backup and restore.
- Advanced search and filtering.
- Web or mobile application.
- Cloud-based data storage.

Conclusion

The Student Management & Academic Record System successfully provides a structured solution for managing student academic information.

The system allows users to manage students and courses, record marks, calculate percentage and grades, manage attendance and generate academic reports.

The project demonstrates the practical application of Core Java, Object-Oriented Programming, collections, validation, exception handling and file handling.

The modular structure of the system also provides a foundation for future enhancements such as database connectivity and a graphical user interface.

REFERENCES

- Java Documentation and Core Java learning resources.
- Object-Oriented Programming course material.
- VITyarthi project guidelines.
- Classroom notes and practical laboratory exercises.